

Reguläre Ausdrücke

Susanne Schaller, MMSc

Andreas Scheibenpflug, MSc

susanne.schaller@fh-hagenberg.at

andreas.scheibenpflug@fh-hagenberg.at



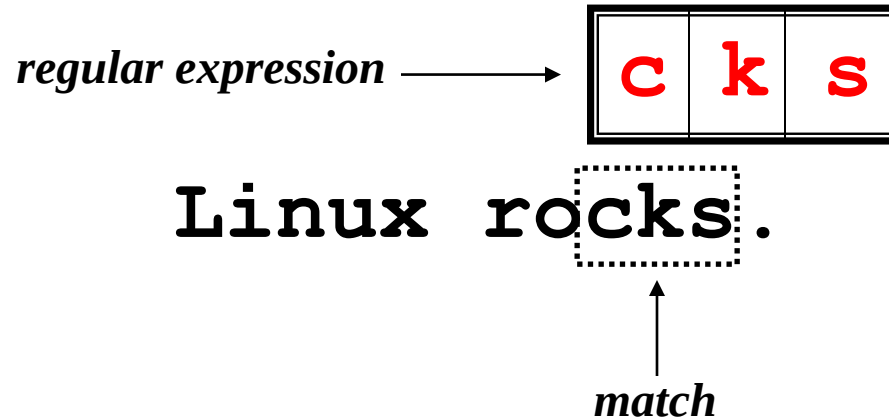
Regular Expressions

- Regular expressions (regexp) (dt. reguläre Ausdrücke), sind eine leistungsstarke, flexible und effiziente Methode zum Definieren von Suchmustern zum Stringvergleich in Texten
- Stellen umfangreiche Notation für den Mustervergleich zur Verfügung:
 - Suche nach bestimmten Zeichenmustern in Texten
 - Validieren von Text, um sicherzustellen, dass er einem vordefinierten Muster entspricht (E-Mail-Adresse, IP-Adressen);
 - Darauf aufsetzend: Extrahieren, Bearbeiten, Ersetzen oder Löschen von Textzeichenfolgen in Editoren bzw. in Programmiersprachen
- Für viele Anwendungen, die mit Zeichenfolgen arbeiten oder große Textblöcke analysieren, sind reguläre Ausdrücke unverzichtbar.
- Einfacher Anwendungsfall von regulären Ausdrücken sind die Wildcards (=Metacharacter) der Shell.

Regular Expressions - Syntax

- Die einfachste regexp ist eine Zeichenkette von einem oder mehreren Symbolen (Buchstaben, Ziffern, Sonderzeichen), nach der in einem Text gesucht wird.
- Falls die Zeichenkette im Text gefunden wird, gibt es eine Übereinstimmung (match).

Regular Expressions: Beispiel



Linux sucks.

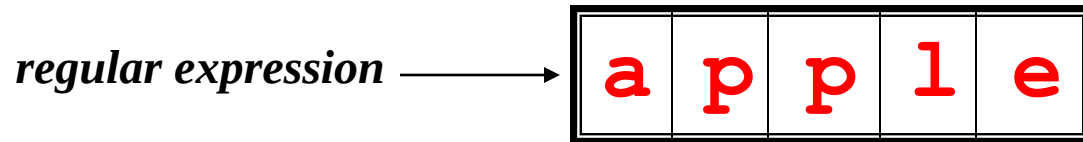
↑
match

Linux is okay.

no match

Regular Expressions: Beispiel

- Eine Übereinstimmung im Text kann mehrfach vorkommen.



Scrapple from the apple.

match 1

match 2

Regular Expressions – Syntax

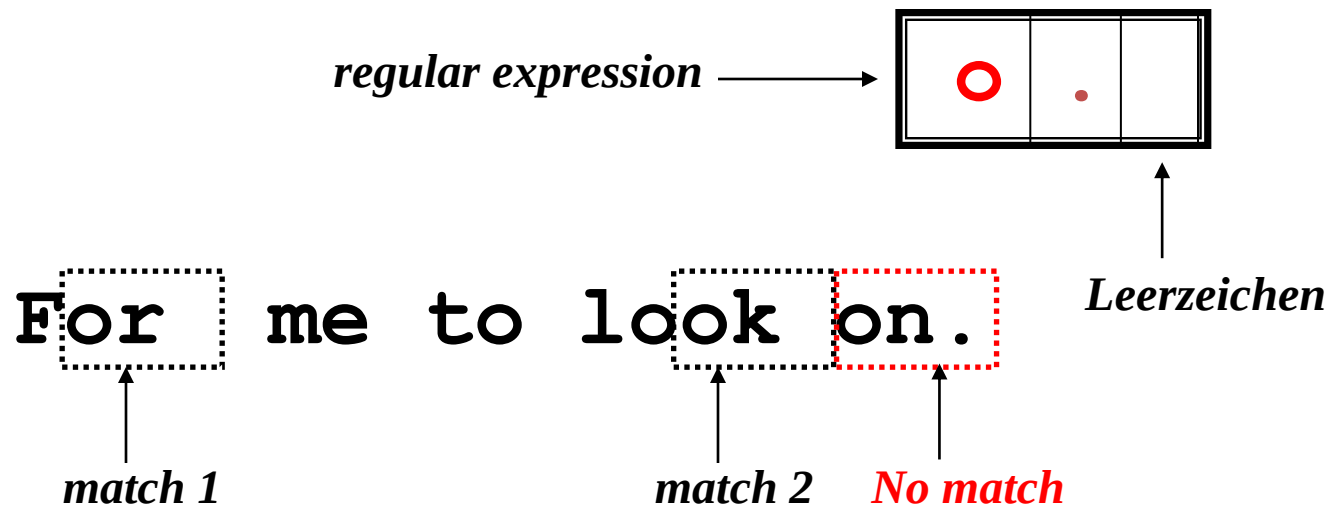
Metazeichen

Metazeichen sind Zeichen, die nicht für sich selbst stehen, sondern eine besondere Bedeutung haben.

Metazeichen	Bedeutung
^	Anker für Anfang einer Zeile
\$	Anker für Ende einer Zeile
? + * { }	Quantifikatoren oder Wiederholungsfaktoren
\	Metazeichen verliert seine spezielle Bedeutung durch Voranstellen des Backslashes \
. []	Zeichenklassen
&	Backreference (hier nicht behandelt)
()	Gruppierung Unterausdrücke
 	Alternative

Regular Expressions - Syntax

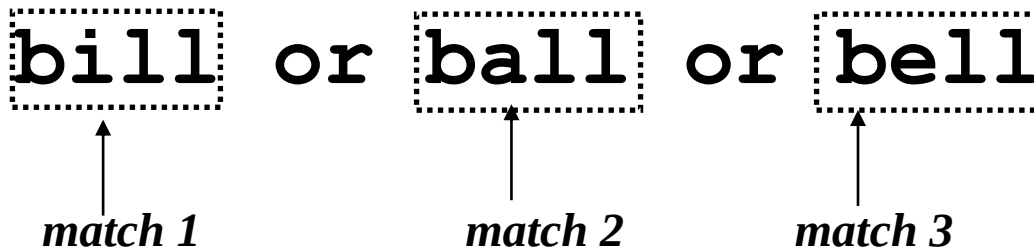
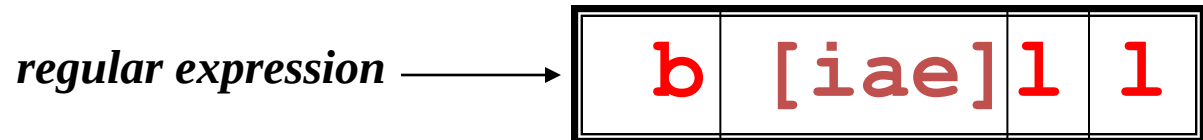
- Das Zeichen **.** ist ein Metazeichen und kann für ein einzelnes beliebiges Zeichen verwendet werden.



Regular Expressions – Syntax

Zeichenklasse

- Zeichenklassen können mit eckigen Klammern **[]** spezifiziert werden.
- Ein einziges Zeichen aus dem Klammerausdruck wird auf Übereinstimmung herangezogen.



Regular Expressions – Syntax

Zeichenklasse negieren

- Eine Zeichenklasse kann auch negiert werden **[^]**.

regular expression →

b	[^ie]	l	l
----------	--------------	----------	----------

bill or **ball** or bell

↑
match

Regular Expressions:

Beispiele Zeichenklassen

- Beispiele
 - `[aeiou]` findet einen Selbstlaut kleingeschrieben
 - `[kK]orn` findet `korn` oder `Korn`
 - `[1-9]` gleichwertig zu `[123456789]`
 - `[abcde]` gleichwertig zu `[a-e]`
 - `[abcde123456789]` gleichwertig zu `[a-e1-9]`
 - Der Bindestrich `-` hat zwar bei der Zeichenauswahl eine Spezialbedeutung, aber nur bei der Angabe eines Bereiches.
 - Schreibt man ihn zu Beginn eines Bereiches würde die regexp `[-1-3]` die Zeichen `-` oder `1` oder `2` oder `3` finden.

Regular Expressions: vordefinierte Zeichenklassen

In vielen Implementierungen sind auch diese Zeichenklassen verfügbar.

Klasse	
[[:alpha:]]	Buchstaben: [[:lower:]] und [[:upper:]]. Dies sind nicht nur lateinische Buchstaben.
[[:lower:]]	Kleinbuchstaben
[[:upper:]]	Großbuchstaben
[[:digit:]]	Ziffern, entspricht [0-9]
[[:alnum:]]	Ziffern und Buchstaben
[[:blank:]]	Leerzeichen und Tabulator
[[:space:]]	verschiedenste Leerzeichen, Tabulatoren, Zeilenumbrüche, Seitenvorschub, Wagenrücklauf, usw.
[[:cntrl:]]	Steuerzeichen (nicht darstellbaren Zeichen)
[[:print:]]	Druckbare Zeichen, entspricht [[:alnum:]], [[:punct:]] und [[:blank:]]
[[:graph:]]	Druckbare Zeichen ohne [[:blank:]]
[[:punct:]]	Satzzeichen und andere Zeichen: , - . ! " # \$ % & ' () { } [] * + / : ; < = > ? @ \ ^ _ ` ~
[[:xdigit:]]	Hexadezimale Ziffern, entspricht [0-9A-Fa-f]

Regular expressions – Anker

- Anker werden für die Markierung eines Zeilenanfangs oder Zeilenendes verwendet (oder beides).
- $\wedge$ bedeutet Zeilenanfang
- $\$$ bedeutet Zeilenende

Regular expressions: Anker

regular expression →



bill

or ball or bell

↑
match

regular expression →



bill or ball or

bell

↑
match

Regular expressions: Anker

Beispiele:

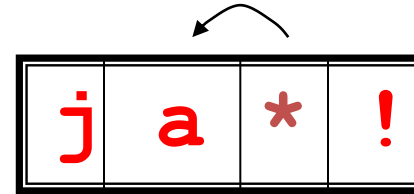
<code>^D</code>	Zeile, die mit D beginnt
<code>D\$</code>	Zeile, die mit D endet
<code>^D\$</code>	Zeile, die nur D enthält
<code>^.\$</code>	Zeile, die nur einen Buchstaben enthält
<code>^...\$</code>	Zeile, die nur drei Buchstaben enthält
<code>^e..</code>	Zeile, die mit e beginnt und mindestens noch zwei Zeichen enthält
<code>^\.</code>	Zeile mit echtem Punkt am Anfang
<code>^\$</code>	leere Zeile

Regular Expressions – Syntax Quantifikatoren

Quantifikator	Beschreibung
?	Der vorangegangene Ausdruck ist optional und wird maximal einmal getroffen
*	Der vorangegangene Ausdruck wird beliebig oft (aber auch keinmal vorgefunden)
+	Der vorangegangene Ausdruck wird mindestens einmal gefunden
{n}	Der vorangegangene Ausdruck wird genau n-mal gefunden
{n,}	Der vorangegangene Ausdruck wird mindestens n-mal oder öfter angetroffen
{n,m}	Der vorangegangene Ausdruck wird mindestens n-mal und maximal m-mal angetroffen
{0,m}	Der vorangegangene Ausdruck wird maximal m-mal angetroffen.

Regular expressions: Quantifikator

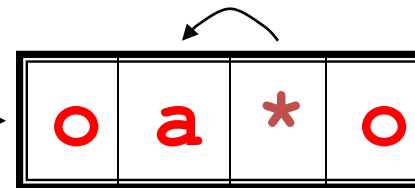
regular expression →



I got money, jaaaaaaaaaaa!

↑
match

regular expression →



Time to look forward

↑
match

.* Beliebige Zeichenfolge auch leere

Regular expressions: Quantifikator

Beispiele:


ca+r

car caaar (mindestens einmal)


ca?r

cr car (keinmal oder einmal)


ca{3}r

caaar (exakt 3x)


ca{3,}r

caaar caaar caaaaar (mindestens dreimal)


ca{3,5}r

caaar caaar caaaaar (genau drei- bis fünfmal)


ca{0,5}r

..... caaar caaar caaaaar (maximal fünfmal)

Regular expressions - Syntax

Unterausdrücke

- Mehrere einzelne regexps können mit Verwendung von **()** zu einem Unterausdruck zusammengefaßt werden.
- Unterausdrücke werden dann wie einzelner Ausdruck behandelt.
 - **a*** findet keine oder beliebig viele Vorkommen des **a**
 - **abc*** findet **ab**, **abc**, **abcc**, **abccc**, ...
 - **(abc) +** findet **abc**, **abcabc**, **abcabcabc**, ...
 - **(abc) {2,3}** findet **abcabc** oder **abcabcabc**

Regular Expression - Syntax

Alternativen

Mehrere Alternativen können durch die Verwendung des **|** Operators unter Verwendung von UnterAusdrücken **()** angegeben werden.

Beispiel:

`I like (dogs|penguins), but not (lions|tigers).`

findet

`I like dogs, but not lions.`

`I like dogs, but not tigers.`

`I like penguins, but not lions.`

`I like penguins, but not tigers.`

Regular Expressions: Beispiele

- *Finde gültige Zeugnisnoten:*
(SGT1|GUT2|BEF3|GEN4|NGD5)
- *Finde gültige Matrikelnummern der SE Studierenden in folgender Form:*
15- letzte beiden Ziffern des Jahres der
 Immatrikulation und -
1- 1- für Einstieg ins WS, 2- für Einstieg ins SS, und -
0307- Studiengangskennzahl und -
005 3 Stellige laufende Nummer
[0-9]{2}-[12]-0307-[0-9]{3}
- *Finde gültige INTEGER Zahlen:*
[+-]?[0-9]+

Regular Expressions: Beispiele

- *Finde Datumsangaben der Form:*
Tag.Monat.Jahr (ohne Plausibilitätsprüfung)
Tag: 1-31
Monat: Jan, Feb, Mar,.....,Dec
Jahr: 1900-2029

[1-3]?[0-9]\. **Ungenau!** (Tag kann nicht null sein!)
(Jan|Feb|Mar|Apr|Mai|Jun|Jul|Aug|Sep|Oct|Nov|Dec)\.
(19[0-9][0-9]|20[0-2][0-9])

Regular Expressions: Bemerkungen

- Viele verschiedene Implementierungen vorhanden
- Varianten abhängig von Programmiersprachen
- Besonders oft verwendet: PCRE (PERL Compatible Regular Expression)
- Bash erlaubt in Kombination mit dem grep Kommando:
 - BRE (basic regular expressions)
 - ERE (extended regular expressions)